

A Grammar of Digital Fabrication Machines

Jasper Tran O’Leary
jaspero@uw.edu
University of Washington
Seattle, Washington, USA

Khang Lee
leekg97@uw.edu
University of Washington
Seattle, Washington, USA

Nadya Peek
nadya@uw.edu
University of Washington
Seattle, Washington, USA

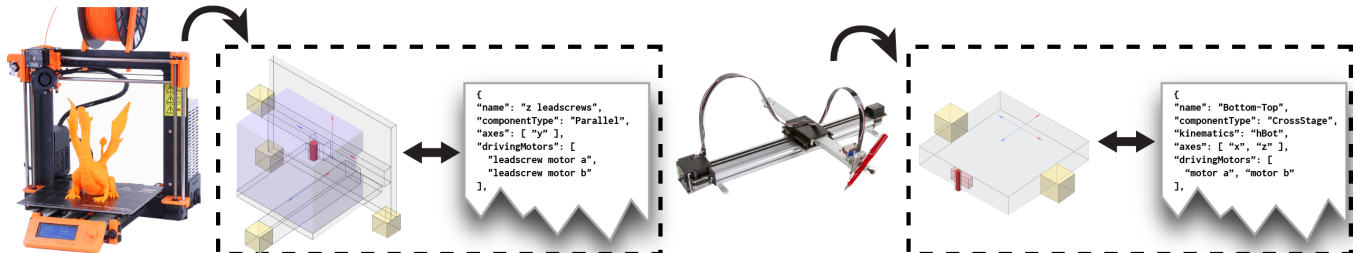


Figure 1: Taxon Concept. We can represent two physical machines—a Prusa i3- MK3S+ 3D printer (leftmost) and an Axidraw V3 pen plotter (center)—as Taxon programs. Taxon programs comprise abstractions in text for each physical machine (right inside dashed boxes) that compile to machine simulations (left inside dashed boxes).

ABSTRACT

Digital fabrication tools for makers have increased access to manufacturing processes such as 3D printing and computer-controlled laser cutting or milling. Despite research advances in novel hardware and software tools for fabrication tasks, there is no formal way to reason about the fabrication machine itself. There is no standard format for representing the high-level features of machines and trade-offs between them; instead, this important information is relegated to folk knowledge. To make machine information explicit, we present Taxon, a machine specification language broad enough to represent many machines, while also allowing for enough expressivity to meaningfully compare and infer performance. We describe and detail the motivation behind the design of Taxon, as well as how Taxon programs compile to a simulation of physical machines. We discuss opportunities for future work in digital fabrication that requires a standard, formalized representation of machines.

CCS CONCEPTS

- Human-centered computing → Interactive systems and tools;
- Applied computing → Computer-aided manufacturing; • Software and its engineering → Domain specific languages.

KEYWORDS

Digital fabrication, programming languages, user interfaces, prototyping

ACM Reference Format:

Jasper Tran O’Leary, Khang Lee, and Nadya Peek. 2021. A Grammar of Digital Fabrication Machines. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts (CHI ’21 Extended Abstracts)*, May 8–13, 2021, Yokohama, Japan. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3411763.3451829>

1 INTRODUCTION

Digital fabrication machines, including 3D printers, laser cutters, CNC mills, drawing machines, laboratory pipetting machines, and more, allow a broadening base of users to program machines to create and modify physical objects. A large infrastructure of software and machine tools alongside increased research in human-computer interaction have unlocked more possibilities for making. At the same time, understanding how to select appropriate machines for a given manufacturing task is crucial for wider adoption and quality control, especially in a decentralized context. However, while most industrial and academic work has focused on new possibilities for making through digital fabrication, there is still a lack of common infrastructure for standardizing and formalizing the machines themselves. Despite the plethora of tools for optimizing parts of the process, or for creating new types of objects, makers will never escape the need to find machines that can manufacture an object to their particular specification. Yet, there exists no unified way of understanding what machines work for which tasks, or how machines vary by their features or properties of motion (*kinematic properties*). *How can we enable formalized classification, simulation, and verification of digital fabrication machine properties?*

To address this, we present Taxon, a prototype specification language for representing digital fabrication machines as programs. Each machine is a program in Taxon, as illustrated in Figure 1. Programs in Taxon are also syntactically valid JSON objects consisting of a list of required properties and values, as shown in Listing 2. The design of the grammar relies on several abstractions that

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CHI ’21 Extended Abstracts, May 8–13, 2021, Yokohama, Japan

© 2021 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-8095-9/21/05...\$15.00
<https://doi.org/10.1145/3411763.3451829>

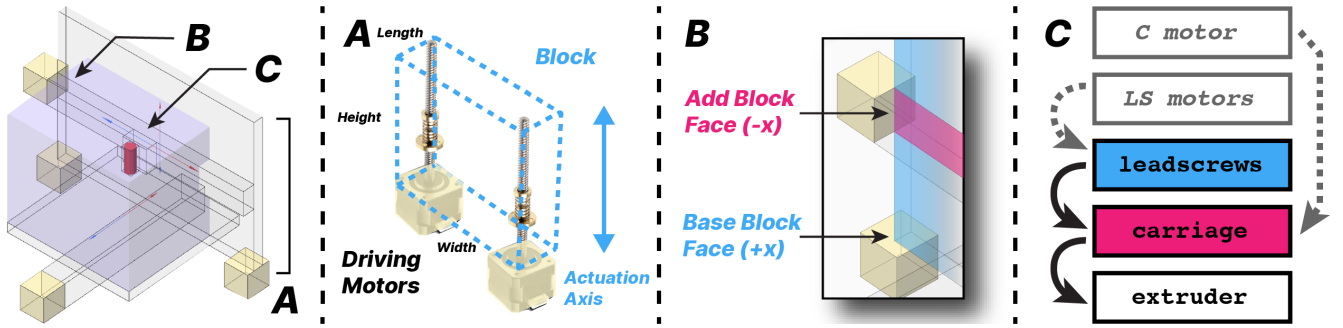


Figure 2: Printer Example Abstractions. A) Two vertical y leadscrews on the physical 3D printer are abstracted into one block object as denoted by the dashed blue box lines, two motors shaded in yellow provide torque to actuate the block up and down on the y-axis; B) a connection between the carriage (red) as an add block onto the y leadscrews (blue) such that the selected faces of each block are coplanar; C) the underlying kinematic chain (arrows indicate parent-child relationships) constructed from the complete set of connections in the printer program where motors (gray, dashed lines) provide torque to blocks. When a block actuates, all descendants move on the former block’s actuation axis.

are high-level enough to support meaningful distinctions between machines, yet specific enough to reasonably model the behavior of the machine in the physical world. First, we describe machines as an assembly of *blocks*, which are bounding-box abstractions for moving parts of the machine. Blocks are related through *connections*, which specify which blocks provide information about how blocks are positioned relative to each other, as well as how movement in one block moves others. Through the blocks and connections, we infer an underlying kinematic chain for the machines which provides information about how the machine moves as a whole to fabricate objects as shown in Figure 2. Finally, programs include details about the machine’s *tool* e.g. whether it is a 3D printer extruder or a pen for plotting, as well as other *machine features* like the total range of motion of the tool (known as the *work envelope*).

Through this information, we can glean important factors such as whether or not a machine is large enough to fabricate a given object, how a machine’s tool might necessitate certain materials, and which machines can fabricate an object without collision. The grammar also provides a means of sorting and searching through machines based on their features; for example, comparing two 3D printers and selecting the one with less cantilevered force (known as the tool’s *structural loop*), among other machine characteristics. As a whole, the grammar lets us represent machines at a coarser grain than representing the machine’s design in a CAD program. This allows us to classify, compare, and search through machines for their high-level features without needing to extract information about the kinematics or drive trains from the CAD model or relying on past experiences with existing machines.

The representation we propose for describing basic characteristics of machines is relatively simple. However, such a representation is necessary for formally reasoning about machines. Despite its simplicity, our representation establishes an initial *taxonomy* of machines, which in turn provides a formal enabling infrastructure for software tools that reason-about and interface-with machines. We aim for Taxon to provide the basis for further research in developing such tools and the standards they rely on.

2 DEFINING A CORE ABSTRACTION: BLOCKS

In other domains, such as information visualization or computer graphics, a grammar is not a description of something, but it *is the thing itself*. For example, in the Vega-Lite grammar, a program in the grammar immediately compiles to and produces the single visualization specified in the program [14]. In our case of digital fabrication machines, a program cannot ever *be* the thing it represents; that thing is a physical object. Instead, programs in the Taxon grammar can *be* simulations (in that they compile to code that simulates machines), but they can only *represent* the physical machines that we are interested in. Thus, it is crucial that the Taxon language have an appropriate abstraction for the physical reality of machines while at the same time not being too low level and verbose.

To address this, we model the machine as an assembly of black boxes we call *blocks*:

Block Abstraction. A block is a bounding-box primitive for physical machine parts that lie inside the bounding box, where blocks can actuate (generate movement) in along the block’s axes according to its physical attributes. The physical machine’s implementation of the block can vary as long as it fulfills the block’s specification in code.

In terms of composing programs, blocks represent the primitives for movement that compose the machine. For example, Listing 1 denotes a carriage in a 3D printer which is connected to two leadscrews, as shown in Figure 2a.

The block object contains properties describing how the physical carriage moves. For Listing 1:

- `axes` tells us that the carriage moves only in the z-axis, providing a single degree of freedom of movement, while `drivingMotors` indicates the motor (which is also declared in the top-level `motors` property, see Listing 2) that supplies this block with torque. This property can contain more than one axis.

```
{
  "name": "carriage",
  "componentType": "LinearStage",
  "axes": [ "z" ],
  "dimensions": {
    "width": 12.5,
    "height": 25,
    "length": 210
  },
  "drivingMotors": [
    "carriage motor"
  ],
  "kinematics": "directDrive",
  "attributes": {
    "driveMechanism": "timingBelt",
    "stepDisplacementRatio": 0.5
  }
}
```

Listing 1: Block denoting the carriage (+y is “up”).

```
{
  "name" : "prusa-i3-mk3",
  "motors" : [ ],
  "blocks" : [ ],
  "connections" : [ ],
  "tool" : { },
  "features" : { }
}
```

Listing 2: Top-level properties of a machine with blank values.

- kinematics describes the transform from the driving motors to block actuation as direct drive. Other types of kinematics including parallel kinematics such as CoreXY are also supported.
- attributes lists physical properties of the drive train that are used with the specified kinematics to calculate displacement. In this case, the sub-property driveMechanism is ‘‘timingBelt’’, which indicates that the carriage uses a timing belt with a pulley (as opposed to a leadscrew or rack-and-pinion drive train) to translate the rotary motion of the motor into linear motion.
- dimensions provides volumetric information for the carriage, which determines its size in the machine simulation shown in Figure 3. Dimensions specify width, height, and length, which are how long the box extends in the x, y, and z directions.
- position lists the location of the centroid of the block in world coordinates. However, only some blocks have explicit positions, while most have positions implicitly defined by their connections or places in the kinematic chain; we describe this further in the next section.

In addition to blocks, a program includes the machine’s motors which are listed in the top-level motors property. Motors provide

```
{
  "baseBlockName": "y leadscrews",
  "baseBlockFace": "+x",
  "baseBlockEnd": "0",
  "addBlockName": "carriage",
  "addBlockFace": "-x",
  "addBlockEnd": "0"
}
```

Listing 3: Connection between two blocks.

the means of actuating each block, for example, the motor named carriage motor connects via a pulley to the timing belt that actuates the carriage along the z-axis. A motor object lists the names of the blocks that it *drives* (provides movement for) when the block actuates, as well as dimensions and possibly position. One block may contain references to multiple motors.

3 CONNECTING BLOCKS TO GENERATE MACHINE KINEMATICS

Once the programmer defines the blocks and motors of the machines, they specify how blocks and motors are connected to each other by declaring connections. Thus, machines are ultimately represented as connected assemblies of blocks. A connection represents a fixed positioning of an *add block* relative to a *base block*. For example, Listing 3 shows the connection in the 3D printer between the y leadscrews as a base block and the carriage as an add block¹. Informally, this means that the carriage block “snaps” to the y leadscrews block such that the positive x-direction face of the y leadscrews block and the negative x-direction face of the carriage block are stuck together. The programmer can further specify at which point on each block’s face—baseBlockEnd and addBlockEnd, respectively—snap together, as shown in Figure 2b.

Declaring a connection implicitly defines the position of the add block to be relative to the base block’s position. As such, the Taxon static checker enforces that no block with an explicit position property also be named as an add block in any connection object, which would cause the system to become overconstrained. One logical way to create a program for a (fully constrained) machine would be to declare blocks and motors for parts of the machine that rest on the ground or are fixed in some way, assign those blocks and motors positions explicitly, and then connect to those explicitly positioned blocks’ subsequent add blocks.

By connecting blocks together via connections, we also obtain an underlying kinematic chain of the machine. Given a block in the kinematic chain, any add blocks of that block (or add blocks of those add blocks recursively) are children of the former block. Besides having volumetric properties, blocks actuate on their axes, which moves all child blocks:

Connection Abstraction. When a block actuates, any child blocks in the kinematic chain move along those axes. The range of movement along a given axis is restricted by the dimensions of the work envelope.

¹Note that we use the convention where +y points in the “up” direction

For example, if we actuate the *y* leadscrews block, any child blocks of *y* leadscrews, including the carriage and its own child blocks recursively, move up or down in the *y*-direction. Blocks that have no defined axes are *non-actuating*; for example, the platform of the 3D printer might be represented as a non-actuating block because, while it itself moves, it does not actuate and provide movement to any other blocks. The *work envelope* is a programmer-specified volume representing where the machine's tool can move. We make the assumption that the total range of movement from an actuating block on a given axis equals the dimension in the work envelope corresponding to that axis; for example, if the height of the work envelope is 400cm, any block that actuates on the *y*-axis can move only within the 400cm.

Given the kinematic chain that the system induces from the blocks and connections, we can simulate machine motion. Topologically, the connections of the chain can be structured as a tree or graph. In the physical world, all movement in digital fabrication machines arises from the turning of motors which then are transferred through the kinematic chain to generation motion of the *tool* e.g. the 3D print head, or any platforms for the fabricated object. Determining the position of the tool is done with *forward kinematics*; for example, turning carriage motor +100 steps given the motor attributes results in 50 mm of carriage movement in the +*z* direction.

In addition, we implemented a browser tool that generates simulated machines from a grammar and accepts either direct motor pulses (forward kinematics) or instructions for movement in the G-Code language (inverse kinematics). For example, the G-Code instruction `G0 X20 Y100 Z0` moves the tool to the coordinate position (20, 100, 0) relative to the location at which the machine was last zeroed (instruction `G92`).

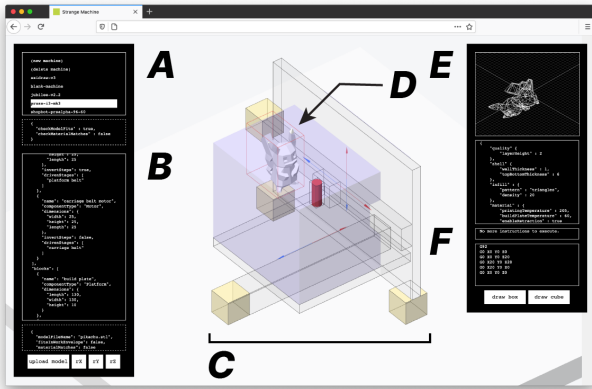


Figure 3: Machine Simulation Interface. The interface runs in a browser based environment and comprises the following parts: A) database of machine programs; B) program viewer; C) interactive machine simulation D) user-uploaded digital model; E) model information display; F) G-Code simulation pane. Digital model “Low-Poly Pikachu” provided by FLOWALISTIK on Thingiverse.

4 DECLARING TOOL AND MACHINE FEATURES

In addition to declaring blocks and their connections, a programmer also declares information about the machine's tool and its other features. The tool refers to the part of the machine whose precise movement with respect to a work piece does the work of manufacturing the desired object, for example, the extruder on a 3D printer, the laser on a laser cutter, or a pen for a pen plotter. The tool top-level property includes the following sub-properties:

- **manufacturingStrategy**: describes the overall purpose or approach of the machine. Examples include 3D printing, milling, laser cutting, and plotting.
- **materialsSupported**: lists the materials that this machine can manufacture with.
- **dimensions**: the dimensions of the tool are important to specify to check for collision detection. This is important e.g. for subtractive manufacturing, which may rely on end mills of varying sizes.

In the features top-level property, the user specifies features that are unrelated to the machine's tool and kinematics, but which are important for decision-making:

- **workEnvelope**: the total volume in which the tool can move, often specified by the manufacturer. Knowing the work envelope is crucial for determining the ranges of motion that blocks can actuate within.
- **buildEnvironment**: denotes a workspace on which the machine is built.
- **vendorInfo**: lists the machine's price, the size of the community that uses the machine, and other information relevant to someone who might build or purchase the machine.

5 MACHINE SIMULATION INTERFACE

5.1 User Interface

We implemented an web interface for loading machine grammars and compiling them into simulations, shown in Figure 3. The interface allows users to load machines grammars from a database, compare differences across machines, and simulate raw motor pulses and G-Code instructions. The interface currently supports basic queries that can be implemented by examining the machine program. For example, the user can upload a desired model (Figure 3e) and show only machines that can manufacture that model based on material constraints and whether the model can fit inside the machine's work envelope (Figure 3d). The interface also supports visual comparisons of machines as shown in Figure ?? . Finally, we can show only machines that fulfill some criteria, such as only showing laser cutters, or machines that include blocks with CoreXY kinematics.

5.2 Compilation Process

To load a machine in the interface, we compile the machine grammar into a simulation. We implemented the machine simulation in THREE.js [4] where the final compiled simulation is a Javascript object containing a complete THREE.js scene and animation system. We informally denote the semantics of our compiler by describing

how each object in a Taxon program is translated into a Javascript object that is rendered in the web browser.

Blocks and Motors: for every block and motor b , instantiate THREE.js mesh m_b as an axis-aligned box whose sizes on the axes $(\Delta x, \Delta y, \Delta z)$ are set as $(d.width, d.height, d.length)$ where d is the dimensions property of b . The centroid of m , c_m , is located either at $b.position$ if defined, otherwise at $(0, 0, 0)$.

Connections: given a 6-tuple connection $(b, b_f, b_e, a, a_f, a_e)$ let b be the base block, a be the add block, and b_f, a_f be the base and add block faces. Let b_e, a_e be the base and add block ends where $b_e, a_e \in \{\pm x, \pm y, \pm z\}$ representing the point on the edge of the corresponding face moving from the centroid of that face on the end's axis. Let K be the set of all child blocks of a in the kinematic tree. Then, translate a using translation T such that b_f, a_f are coplanar and b_e, a_e occupy the same point. Finally, apply T to all $k_i \in K$.

Features: for the build environment e , instantiate an opaque mesh m_e with width and length defined in `buildEnvironment` such that $c_{m_e} = (0, 0 - h/2, 0)$ for some constant build environment height h . For the work envelope w , instantiate a translucent mesh m_w such that the negative y-axis direction face of m_w is coplanar to the positive y-axis direction face of m_{b_p} where $b_p \ni \{“isPlatform” : true\}$ if and only if there exists such a b_p . Otherwise m_w is coplanar to the positive y-axis direction face of m_e .

6 RELATED WORK

Much of prior research in HCI and graphics has proposed new systems for making with digital fabrication tools, while a smaller subset of work has investigated challenges that arise during the process. For example, Yildirim et al. interviewed professionals needs and challenges with these tools, noting that “negotiation of [fabrication] trade-offs” was a crucial part of a maker’s skill set, yet was not facilitated by most tools [20]. Hudson et al. studied challenges to 3D printing for newcomers, noting that one way to reduce barriers would be “to ‘weave in’ expert tips, advice, and explanations throughout the printing workflow” [9]. Torres et al. examined strategies or rituals that makers would adopt to account for the possibilities of failure with fabrication machines [17]. Some design researchers, such as Andersen et al. [2], Albaugh et al. [1], and Devendorf et al. [6] have foregrounded the experience of blending human craft with machine precision, along with developing mindsets that account for the tensions between the two. From a high level, these studies call attention to the fact that the capabilities and limitations of a given machine must be accounted for at all parts of the making process. Rather than rely solely on a maker’s expertise for this, Taxon formalizes these trade-offs and characteristics explicitly as programs, thus enabling machine thinking at throughout the process.

To design Taxon, we drew from prior work in programming languages research as well as work in information visualization research. In particular, we adopted the strategy used by the Vega and Vega-Lite grammars where all information required to generate lower-level code for a visualization (in D3.js) is represented declaratively in JSON format [3, 14, 15]. Formalizing visualization

properties in this way enables automated recommendation of visualization chart types [19] and formalizing design best practices as constraints [10]. Building on interactivity, Hempel et al. built a system for authorizing SVG graphics where programmatic representations of the SVG were synchronized with direct manipulation edits performed on the graphic [8]. Moving into the realm of digital fabrication, Nandi et al. contributed formalized languages of CAD models and triangle meshes that enabled decompilation of mesh models to CAD programs [11]. Nandi and colleagues also contributed a system for simplifying complex CAD programs into simpler ones via equality saturation [12].

The task of creating a grammar of digital fabrication machines is difficult because these machines exist as physical objects with unmeasured details, not output programs that can be formalized exactly. To develop abstractions for representing machines at balance between coarse- and fine-grain, we drew on machine design principles as formulated by Slocum [16], as well as abstractions used in the common robotics middleware ROS [7]. We also drew from Tran O’Leary and Peek’s initial work that formulated workflows involving digital fabrication tools into programs [18], as well as Desai et al.’s abstractions for interactive robot design [5]. Overall, our goal was to leverage prior techniques to formalize abstractions as grammars and apply these techniques to the foundational component of digital fabrication: the machine itself.

7 FUTURE WORK: AFFORDANCES OF FORMAL REPRESENTATION

A main goal of creating a formal representation of digital fabrication machines is to enable future tools that require the characteristics of each machine. Taxon has limitations and cannot describe all possible types of machines. However, we believe that even in its preliminary state, we would be able to use Taxon to develop tools to help people understand, compare, and build upon digital fabrication machines. Some examples of such tools include:

Querying for Machines. similar to how the Voyager tool allowed for structured exploration of visualizations based on their grammars [19], we could allow for structured exploration of machine options given a certain fabrication task. Makers would be able to explore trade-offs between speed, precision, cost, and other factors that are influenced by machine choices for the task.

Optimizing Instructions for Machine Kinematics. in most machines, dedicated control software translates G-Code instructions into motor pulses so as to optimize the movement of the machine based on its distribution of mass. Given a Taxon program that allows us to infer volumes, masses (in the future), and kinematics, it would be possible to optimize G-Code instructions for any machine. One example of optimization is *input shaping*, which involves generating signals that cancel vibrations associated with moving machine parts [13].

8 CONCLUSION

We contributed and described the design of Taxon, a grammar for formally specifying abstract machine properties, that enables description, comparison, and simulation of the physical machines.

The promise of digital fabrication lies in how it might allow new practitioners to create objects that uniquely suit their own contexts. The capabilities and limitations of machines will always be an important factor in both making and research on making, and they ought to be made explicit. We aim for Taxon to become a useful standard in fabrication research and practice: to allow researchers to integrate machine-level concerns into novel tools, and to provide makers with infrastructure to guide their manufacturing processes.

ACKNOWLEDGMENTS

We would like to thank Chandrakana Nandi, Gabrielle Benabdallah, and Blair Subbaraman for their feedback. This work was supported by the Alfred P. Sloan Foundation.

REFERENCES

- [1] Lea Albaugh, Scott E. Hudson, Lining Yao, and Laura Devendorf. 2020. Investigating Underdetermination Through Interactive Computational Handweaving. In *Proceedings of the 2020 ACM Designing Interactive Systems Conference (DIS '20)*. Association for Computing Machinery, New York, NY, USA, 1033–1046. <https://doi.org/10.1145/3357236.3395538>
- [2] Kristina Andersen, Ron Wakkary, Laura Devendorf, and Alex McLean. 2019. Digital crafts-machine-ship: creative collaborations with machines. *Interactions* 27, 1 (Dec. 2019), 30–35. <https://doi.org/10.1145/3373644>
- [3] M. Bostock, V. Ogievetsky, and J. Heer. 2011. D³ Data-Driven Documents. *IEEE Transactions on Visualization and Computer Graphics* 17, 12 (Dec. 2011), 2301–2309. <https://doi.org/10.1109/TVCG.2011.185> Conference Name: IEEE Transactions on Visualization and Computer Graphics.
- [4] Ricardo Cabello. 2014. three.js - Javascript 3D library. <https://threejs.org/>
- [5] Ruta Desai, Ye Yuan, and Stelian Coros. 2017. Computational abstractions for interactive design of robotic devices. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*. 1196–1203. <https://doi.org/10.1109/ICRA.2017.7989143>
- [6] Laura Devendorf, Abigail De Kosnik, Kate Mattingly, and Kimiko Ryokai. 2016. Probing the Potential of Post-Anthropocentric 3D Printing. In *Proceedings of the 2016 ACM Conference on Designing Interactive Systems (DIS '16)*. ACM, New York, NY, USA, 170–181. <https://doi.org/10.1145/2901790.2901879>
- [7] Willow Garage. 2019. Unified Robot Description Format - ROS Wiki. <http://wiki.ros.org/urdf>
- [8] Brian Hempel, Justin Lubin, and Ravi Chugh. 2019. Sketch-n-Sketch: Output-Directed Programming for SVG. In *Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology (UIST '19)*. Association for Computing Machinery, New Orleans, LA, USA, 281–292. <https://doi.org/10.1145/3332165.3347925>
- [9] Scott E. Hudson. 2014. Printing Teddy Bears: A Technique for 3D Printing of Soft Interactive Objects. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '14)*. ACM, New York, NY, USA, 459–468. <https://doi.org/10.1145/2556288.2557338>
- [10] D. Moritz, C. Wang, G. L. Nelson, H. Lin, A. M. Smith, B. Howe, and J. Heer. 2019. Formalizing Visualization Design Knowledge as Constraints: Actionable and Extensible Models in Draco. *IEEE Transactions on Visualization and Computer Graphics* 25, 1 (Jan. 2019), 438–448. <https://doi.org/10.1109/TVCG.2018.2865240> Conference Name: IEEE Transactions on Visualization and Computer Graphics.
- [11] Chandrakana Nandi, James R. Wilcox, Pavel Panchekha, Taylor Blau, Dan Grossman, and Zachary Tatlock. 2018. Functional Programming for Compiling and Decompiling Computer-aided Design. *Proc. ACM Program. Lang.* 2, ICFP (July 2018), 99:1–99:31. <https://doi.org/10.1145/3236794>
- [12] Chandrakana Nandi, Max Willsey, Adam Anderson, James R. Wilcox, Eva Darulova, Dan Grossman, and Zachary Tatlock. 2020. Synthesizing structured CAD models with equality saturation and inverse transformations. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2020)*. Association for Computing Machinery, New York, NY, USA, 31–44. <https://doi.org/10.1145/3385412.3386012>
- [13] Rush D. Robinson (Ed.). 2002. *Flexible robot dynamics and controls*. Number 19 in IFSR international series on systems science and engineering. Kluwer Academic/Plenum Publishers, New York.
- [14] Arvind Satyanarayan, Dominik Moritz, Kanit Wongsuphasawat, and Jeffrey Heer. 2017. Vega-Lite: A Grammar of Interactive Graphics. *IEEE Transactions on Visualization and Computer Graphics* 23, 1 (Jan. 2017), 341–350. <https://doi.org/10.1109/TVCG.2016.2599030>
- [15] Arvind Satyanarayan, Kanit Wongsuphasawat, and Jeffrey Heer. 2014. Declarative interaction design for data visualization. In *Proceedings of the 27th annual ACM symposium on User interface software and technology (UIST '14)*. Association for Computing Machinery, New York, NY, USA, 669–678. <https://doi.org/10.1145/2642918.2647360>
- [16] Alexander H. Slocum. 2008. FUNdAMENTALS of Design. <http://pergatory.mit.edu/resources/fundamentals.html>
- [17] Cesar Torres, Sarah Sterman, Molly Nicholas, Richard Lin, Eric Pai, and Eric Paulos. 2018. Guardians of Practice: A Contextual Inquiry of Failure-Mitigation Strategies within Creative Practices. In *Proceedings of the 2018 Designing Interactive Systems Conference (DIS '18)*. Association for Computing Machinery, New York, NY, USA, 1259–1267. <https://doi.org/10.1145/3196709.3196795>
- [18] Jasper Tran O'Leary and Nadya Peek. 2019. Machine-o-Matic: A Programming Environment for Prototyping Digital Fabrication Workflows. In *The Adjunct Publication of the 32nd Annual ACM Symposium on User Interface Software and Technology (UIST '19)*. ACM, New York, NY, USA, 134–136. <https://doi.org/10.1145/3332167.3356897> event-place: New Orleans, LA, USA.
- [19] K. Wongsuphasawat, D. Moritz, A. Anand, J. Mackinlay, B. Howe, and J. Heer. 2016. Voyager: Exploratory Analysis via Faceted Browsing of Visualization Recommendations. *IEEE Transactions on Visualization and Computer Graphics* 22, 1 (Jan. 2016), 649–658. <https://doi.org/10.1109/TVCG.2015.2467191> Conference Name: IEEE Transactions on Visualization and Computer Graphics.
- [20] Nur Yildirim, James McCann, and John Zimmerman. 2020. Digital Fabrication Tools at Work: Probing Professionals' Current Needs and Desired Futures. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems (CHI '20)*. Association for Computing Machinery, New York, NY, USA, 1–13. <https://doi.org/10.1145/3313831.3376621>